The Linux Commands Handbook

100+ Essential Commands for Everyday Use and System Administration

A Practical Reference for Students, Developers, and DevOps Engineers

Compatible with Ubuntu, Debian, Fedora, CentOS Stream, and Arch Linux.

About This Handbook

This handbook is a practical, no-fluff reference to the Linux commands you will actually use — from your first day navigating a terminal to the commands senior engineers reach for daily in production. Every command is compatible with modern distributions including Ubuntu, Debian, Fedora, CentOS Stream, and Arch Linux, with distribution-specific notes called out wherever behavior differs.

Each chapter groups related commands into a quick-reference table, then goes deeper on the one or two commands you'll use most, with full syntax, real examples, expected output, common mistakes, and pro tips. The book closes with a 50-command cheat sheet and an interview-preparation chapter for anyone using this to study for a role.

Table of Contents

CH 1 Introduction to Linux

Linux is a free, open-source, Unix-like operating system kernel that powers everything from smartphones to supercomputers to nearly every web server on the internet. A 'distribution' (distro) packages the kernel with a set of tools, a package manager, and often a desktop environment.

Quick Reference

Distribution	Family	Notes
`Ubuntu / Debian`	`Debian-based`	APT package manager, most beginner-friendly, widest community support
`Fedora`	`Red Hat-based`	DNF package manager, cutting-edge packages, upstream for RHEL
`CentOS Stream`	`Red Hat-based`	YUM/DNF, rolling preview of RHEL, common in enterprise servers
`Arch Linux`	`Independent`	Pacman package manager, rolling release, minimal and highly configurable

Summary

Every distribution shares the same Linux kernel and the same core commands. The differences you'll mostly notice are the package manager and default configuration — the commands in this book work the same way everywhere.

Practice Exercises

- Open a terminal and run `uname -a`. Identify the kernel version and architecture.
- Look up which distribution family your own machine (or VM) belongs to.

CH 2 Linux File System Structure

Unlike Windows, Linux has a single unified directory tree starting at root (/), with no drive letters. Every file, device, and partition is mounted somewhere under this tree.

Quick Reference

Path	Purpose
`/`	Root directory — the top of the entire file system tree
`/bin, /usr/bin`	Essential user command binaries (ls, cp, bash, etc.)
`/sbin, /usr/sbin`	System administration binaries (fdisk, iptables, etc.)
`/etc`	System-wide configuration files (plain text)
`/home`	Personal directories for each user (e.g. /home/alice)
`/root`	Home directory for the root (superuser) account
`/var`	Variable data: logs, mail spools, caches
`/tmp`	Temporary files, typically cleared on reboot
`/opt`	Optional third-party software packages

Path	Purpose
`/usr`	User-installed applications, libraries, and documentation
`/lib, /usr/lib`	Shared libraries needed by binaries in /bin and /sbin
`/dev`	Device files representing hardware (disks, terminals, etc.)
`/proc`	Virtual filesystem exposing kernel and process information
`/mnt, /media`	Mount points for external and removable storage
`/boot`	Kernel, initrd, and bootloader files

Summary

Knowing this layout matters practically: logs live under /var/log, configs under /etc, and temporary scratch space under /tmp. Most day-to-day work happens in /home and, for admins, /etc and /var.

Practice Exercises

- Run `ls /etc | head -20` and identify three configuration files you recognize.
- Run `df -h /` to see how much space the root partition has.

CH 3 Navigation Commands

These are the commands you'll type more than any others — moving around the file system and confirming where you are.

Quick Reference

Command	Syntax	Description
pwd	`pwd`	Print the current working directory
cd	`cd [directory]`	Change the current directory
ls	`ls [options] [path]`	List directory contents
tree	`tree [path]`	Show a directory as a visual tree (may need installing)
pushd / popd	`pushd dir / popd`	Jump to a directory while remembering the previous one
cd -	`cd -`	Return to the previous directory
cd ~	`cd ~`	Go to the current user's home directory

Spotlight: cd

Syntax

```
cd [directory]
```

Option	Meaning
`(no argument)`	Go to your home directory
`..`	Move up one directory level
`-`	Toggle back to the previous directory

Example

```
cd /var/log
```

Expected Output

```
(prompt changes to reflect /var/log, no output on success)
```

Use Case: Moving between project folders, log directories, or config locations quickly.

> **Common Mistake:** Forgetting that `cd` with no arguments always goes home — a common surprise when scripting.

> **Pro Tip:** Chain `cd -` to bounce between two directories without retyping full paths.

Spotlight: ls

Syntax

```
ls [options] [path]
```

Option	Meaning
`-l`	Long format: permissions, owner, size, date
`-a`	Show hidden files (those starting with a dot)
`-h`	Human-readable sizes (used with -l)
`-t`	Sort by modification time, newest first

Example

```
ls -lah /home/alice
```

Expected Output

```
drwxr-xr-x 5 alice alice 4.0K Jul 20 09:14 .
-rw-r--r-- 1 alice alice 220 Jul 20 09:14 .bash_logout
```

Use Case: Checking file permissions and sizes before copying, deleting, or sharing files.

> **Common Mistake:** Assuming `ls` shows hidden files by default — it doesn't, you need -a.

> **Pro Tip:** Alias `ll` to `ls -lah` in your .bashrc for a faster daily workflow.

Summary

Master `cd`, `ls`, and `pwd` first — they form the backbone of every terminal session you'll ever have.

Practice Exercises

- Navigate to /var/log using `cd`, then return to your home directory in one command.
- List all files in your home directory, including hidden ones, sorted by modification time.

CH 4 File and Directory Management

Creating, copying, moving, and deleting files and directories — the basic file operations behind almost every task.

Quick Reference

Command	Syntax	Description
touch	`touch file`	Create an empty file, or update its timestamp
mkdir	`mkdir [-p] dir`	Create a new directory (-p creates parent dirs too)
rm	`rm [-r] [-f] file`	Remove files or directories
rmdir	`rmdir dir`	Remove an empty directory

Command	Syntax	Description
cp	`cp [-r] src dest`	Copy files or directories
mv	`mv src dest`	Move or rename files and directories
ln	`ln [-s] target link`	Create a hard link, or symbolic link with -s
stat	`stat file`	Show detailed metadata about a file

Spotlight: cp

Syntax

```
cp [options] source destination
```

Option	Meaning
`-r`	Recursive — required to copy directories
`-v`	Verbose — print each file as it's copied
`-i`	Interactive — prompt before overwriting
`-u`	Update — only copy when source is newer

Example

```
cp -riv project/ project_backup/
```

Expected Output

```
'project/config.yml' -> 'project_backup/config.yml'
'project/main.py' -> 'project_backup/main.py'
```

Use Case: Backing up a project folder before making risky changes.

> **Common Mistake:** Running `cp file1 file2 file3 dest` when `dest` doesn't exist as a directory — the last file silently overwrites instead of copying into a folder.

> **Pro Tip:** Always use -i when copying over files you care about; it costs one keystroke and prevents accidents.

Spotlight: rm

Syntax

```
rm [options] file
```

Option	Meaning
`-r`	Recursive — required to remove directories
`-f`	Force — skip confirmation and ignore missing files
`-i`	Interactive — confirm before each deletion

Example

```
rm -ri old_logs/
```

Expected Output

```
rm: descend into directory 'old_logs'? y
rm: remove regular file 'old_logs/app.log'? y
```

Use Case: Cleaning up temporary files, old logs, or build artifacts.

> **Common Mistake:** Running `rm -rf /` or `rm -rf *` from the wrong directory — there is no undo, no recycle bin.

> **Pro Tip:** Before any `rm -rf`, run the equivalent `ls` first to see exactly what will be deleted.

Summary

These commands are permanent and unforgiving — Linux has no trash can by default. Get comfortable with `ls` before every `rm`.

Practice Exercises

- Create a directory structure `project/src/utils` in a single `mkdir` command.
- Copy that entire structure to `project_backup` while preserving all files.

CH 5 Viewing File Contents

Reading a file's contents without opening a full editor is one of the most frequent things you'll do, especially with logs.

Quick Reference

Command	Syntax	Description
cat	`cat file`	Print entire file contents to the terminal
less	`less file`	View file contents one screen at a time (scrollable)
more	`more file`	Older, simpler pager than less
head	`head [-n N] file`	Show the first N lines (default 10)
tail	`tail [-n N] file`	Show the last N lines (default 10)
tail -f	`tail -f file`	Follow a file live as new lines are appended
nl	`nl file`	Print file contents with line numbers
tac	`tac file`	Print a file in reverse line order

Spotlight: less

Syntax

```
less [options] file
```

Option	Meaning
`/pattern`	Search forward for a pattern inside less
`q`	Quit less
`g / G`	Jump to the start / end of the file

Example

```
less /var/log/syslog
```

Expected Output

```
(opens an interactive, scrollable view of the file)
```

Use Case: Reading large log files without loading the whole thing into your terminal buffer.

> **Common Mistake:** Using `cat` on a multi-thousand-line file and losing the useful part in a wall of scrollback.

> **Pro Tip:** Inside less, press `/errorterm` and Enter to jump straight to matches, then `n` for the next one.

Spotlight: tail -f

Syntax

```
tail -f [-n N] file
```

Option	Meaning
`-f`	Follow mode — keeps printing new lines as they're written

`-n N`	Start by showing the last N lines before following

Example

```
tail -f -n 50 /var/log/nginx/access.log
```

Expected Output

```
(shows the last 50 lines, then streams new ones live as requests come in)
```

Use Case: Watching a live application or web server log while reproducing a bug.

> **Common Mistake:** Forgetting to Ctrl+C out of `tail -f` — it will run forever until stopped.

> **Pro Tip:** Combine with grep: `tail -f app.log | grep ERROR` to watch only the lines you care about.

Summary

Use `cat` for short files, `less` for anything long, and `tail -f` for anything live.

Practice Exercises

- View the first 15 lines of `/etc/passwd` using `head`.
- Open any log file in `less` and search for the word 'error'.

CH 6 Searching Files

Locating files by name, type, or metadata across the file system — distinct from searching inside file contents, which is covered in the next chapter.

Quick Reference

Command	Syntax	Description
`find`	`find [path] [expr]`	Search for files/directories matching criteria
`locate`	`locate name`	Fast search using a prebuilt file index
`updatedb`	`updatedb`	Rebuild the index used by locate
`which`	`which command`	Show the full path of a command that would run
`whereis`	`whereis command`	Show binary, source, and man page locations
`find -name`	`find . -name '*.log'`	Find files matching a name pattern
`find -exec`	`find . -name '*.tmp' -exec rm {} \;`	Run a command on each match

Spotlight: find

Syntax

```
find [path] [expression]
```

Option	Meaning
`-name 'pattern'`	Match by filename (case-sensitive)
`-iname 'pattern'`	Match by filename, case-insensitive
`-type f / -type d`	Restrict to files / directories
`-mtime -7`	Modified within the last 7 days
`-size +100M`	Larger than 100 megabytes

Example

```
find /var/log -type f -name '*.log' -mtime -1
```

Expected Output

```
/var/log/syslog
/var/log/auth.log
```

Use Case: Finding recently changed log files, or large files eating up disk space.

> **Common Mistake:** Forgetting to quote wildcard patterns (`-name *.log` instead of `-name '*.log'`), which lets the shell expand the pattern before find sees it.

> **Pro Tip:** Pipe find into `xargs` instead of `-exec` for better performance on large result sets: `find . -name '*.tmp' | xargs rm`.

Summary

Use `locate` for a quick, fast guess by name; use `find` when you need precision — by type, size, age, or to act on the results.

Practice Exercises

- Find every `.conf` file under `/etc` modified in the last 30 days.
- Use `which` to confirm exactly which `python3` binary your shell would run.

CH 7 Text Processing Commands

The Unix philosophy in action: small, composable tools for filtering, transforming, and summarizing text — usually chained together with pipes.

Quick Reference

Command	Syntax	Description
grep	`grep [options] pattern file`	Search text using patterns/regular expressions
sed	`sed 's/old/new/' file`	Stream editor for find-and-replace and transforms
awk	`awk '{ print $1 }' file`	Pattern-driven text processing, column-aware
cut	`cut -d',' -f1 file`	Extract columns/fields from each line
sort	`sort [options] file`	Sort lines of text
uniq	`uniq [options] file`	Filter or count adjacent duplicate lines
wc	`wc [-l|-w|-c] file`	Count lines, words, or characters
tr	`tr 'a-z' 'A-Z'`	Translate or delete characters
diff	`diff file1 file2`	Show differences between two files

Spotlight: grep

Syntax

```
grep [options] pattern file
```

Option	Meaning
`-i`	Case-insensitive search
`-r`	Recursive search through directories

`-n`	Show line numbers of matches
`-v`	Invert match — show lines that don't match
`-E`	Extended regular expressions

Example

```
grep -rn 'TODO' ./src
```

Expected Output

```
./src/app.py:42:# TODO: handle empty input
./src/utils.py:7:# TODO: refactor this
```

Use Case: Searching an entire codebase for a function name, error string, or TODO marker.

> **Common Mistake:** Forgetting -r when searching a directory — grep only checks the exact file/path given.

> **Pro Tip:** Combine -v and a pipe to filter noise: `grep ERROR app.log | grep -v DEBUG`.

Spotlight: awk

Syntax

```
awk 'pattern { action }' file
```

Option	Meaning
`$1, $2 ...`	Reference the 1st, 2nd, etc. column of each line
`-F','`	Set the field separator (here, a comma)
`NR`	Built-in variable for the current line/record number

Example

```
awk -F',' '{ print $1, $3 }' users.csv
```

Expected Output

```
alice admin
bob developer
```

Use Case: Pulling specific columns out of structured log files or CSV exports.

> **Common Mistake:** Trying to do complex multi-line logic in a one-liner instead of writing an awk script file for anything beyond simple column extraction.

> **Pro Tip:** `awk '{ print $NF }'` always prints the last column, regardless of how many columns each line has.

Summary

grep finds, sed transforms, awk processes columns, and sort/uniq/wc summarize — chaining these together with pipes is one of the most powerful skills in this entire book.

Practice Exercises

- Count how many lines in `/var/log/syslog` contain the word 'error' (case-insensitive).
- Use `sort` and `uniq -c` together to count how many times each unique word appears in a file.

CH 8 File Permissions and Ownership

Linux enforces access control through a permission model of owner, group, and others, each with read, write, and execute bits.

Quick Reference

Command	Syntax	Description
chmod	`chmod [mode] file`	Change file permissions
chown	`chown user:group file`	Change file owner and/or group
chgrp	`chgrp group file`	Change just the group of a file
umask	`umask [mode]`	Set default permissions for newly created files
sudo	`sudo command`	Run a command as another user (usually root)
su	`su [user]`	Switch to another user's shell

Spotlight: chmod

Syntax

```
chmod [mode] file
```

Option	Meaning
`u/g/o + r/w/x`	Symbolic mode: user/group/other + read/write/execute
`755, 644, etc.`	Numeric mode: read=4, write=2, execute=1, summed per role
`-R`	Apply recursively to a directory and its contents

Example

```
chmod 755 deploy.sh
```

Expected Output

```
(no output; run `ls -l deploy.sh` to confirm: -rwxr-xr-x)
```

Use Case: Making a script executable, or locking down a config file to owner-only access.

> **Common Mistake:** Running `chmod 777` on a file 'to make it work' — this grants write access to every user on the system and is almost always the wrong fix.

> **Pro Tip:** Remember the numeric shortcut: 4=read, 2=write, 1=execute. 755 = rwx for owner, r-x for group and others.

Spotlight: chown

Syntax

```
chown [options] user:group file
```

Option	Meaning
`-R`	Apply recursively to a directory and its contents
`user:group`	Set both owner and group in one call
`user`	Set just the owner, leaving the group unchanged

Example

```
sudo chown -R www-data:www-data /var/www/myapp
```

Expected Output

```
(no output on success)
```

Use Case: Handing a directory's ownership to the service account (like a web server) that needs to read and write it.

> **Common Mistake:** Changing ownership without sudo when you're not the current owner, which silently fails or errors with 'Operation not permitted'.

> **Pro Tip:** Use `chown user: file` (with a trailing colon, no group) to change the owner while keeping the current group.

Summary

Permissions are read (view/list), write (modify), and execute (run/enter), applied to owner, group, and others respectively — always the minimum permission needed, never 777.

Practice Exercises

- Create a script file and make it executable for the owner only.
- Check the numeric permission value for a file that is rwxr-xr-- .

CH 9 Compression and Archiving

Bundling multiple files into one archive, and shrinking file size for storage or transfer.

Quick Reference

Command	Syntax	Description
tar	`tar [options] archive.tar files`	Bundle files into a single archive
gzip	`gzip file`	Compress a file, producing file.gz
gunzip	`gunzip file.gz`	Decompress a .gz file
zip	`zip archive.zip files`	Create a .zip archive
unzip	`unzip archive.zip`	Extract a .zip archive
bzip2	`bzip2 file`	Compress using the bzip2 algorithm (higher ratio, slower)
xz	`xz file`	Compress using xz (very high ratio, common for source tarballs)

Spotlight: tar

Syntax

```
tar [options] -f archive.tar files
```

Option	Meaning
`-c`	Create a new archive
`-x`	Extract an existing archive
`-z`	Filter through gzip (produces/reads .tar.gz)
`-v`	Verbose — list files as they're processed
`-f`	Specify the archive filename (almost always required)

Example

```
tar -czvf backup.tar.gz /home/alice/project
```

Expected Output

```
home/alice/project/
home/alice/project/main.py
home/alice/project/README.md
```

Use Case: Backing up a directory, or preparing a codebase for transfer to another server.

> **Common Mistake:** Forgetting -f and its filename argument, which causes tar to write the archive to standard output instead of a file.

> **Pro Tip:** Remember it with 'create/extract, ze, verbose, file': czvf to make an archive, xzvf to open one.

Summary

tar bundles files together (with optional gzip compression via -z); zip is more portable to non-Linux systems; xz gives the best compression ratio at the cost of speed.

Practice Exercises

- Create a compressed tar.gz archive of your home directory's Documents folder.
- Extract that archive into a new directory called `restored/`.

CH 10 Process Management

Every running program is a process with a unique ID (PID). These commands let you inspect, control, and stop them.

Quick Reference

Command	Syntax	Description
ps	`ps aux`	List currently running processes
top	`top`	Live, auto-refreshing view of processes and resource usage
htop	`htop`	Improved, interactive version of top (may need installing)
kill	`kill [-signal] PID`	Send a signal (usually terminate) to a process
killall	`killall name`	Kill all processes matching a name
bg / fg	`bg / fg`	Resume a job in the background / foreground
jobs	`jobs`	List background jobs in the current shell
nice / renice	`nice -n 10 command`	Start (or change) a process's scheduling priority
nohup	`nohup command &`	Run a command immune to hangups after logout

Spotlight: ps

Syntax

```
ps [options]
```

Option	Meaning
`aux`	Show all processes, all users, in a detailed BSD-style format
`-ef`	Show all processes in a full System V-style format
`--sort=-%mem`	Sort output by memory usage, descending

Example

```
ps aux --sort=-%mem | head -5
```

Expected Output

```
USER PID %CPU %MEM COMMAND
root 1122 0.3 4.1 /usr/bin/mysqld
web 980 1.2 2.8 node server.js
```

Use Case: Finding which process is consuming the most memory or CPU before deciding what to kill or restart.

Common Mistake: Confusing PID (process ID, unique) with PPID (parent process ID) when trying to kill the right process.

Pro Tip: Pipe `ps aux` into grep to find a specific process: `ps aux | grep nginx`.

Spotlight: kill

Syntax

```
kill [-signal] PID
```

Option	Meaning
`-15 (default, TERM)`	Politely ask the process to shut down and clean up
`-9 (KILL)`	Force-terminate immediately, no cleanup
`-1 (HUP)`	Ask the process to reload its configuration

Example

```
kill -9 4021
```

Expected Output

```
(no output on success; the process disappears from `ps`)
```

Use Case: Stopping a hung or unresponsive application that isn't responding to a normal close.

Common Mistake: Reaching for `kill -9` as the first option — it skips cleanup and can corrupt open files; try plain `kill` first.

Pro Tip: Use `kill -0 PID` to check whether a process exists at all, without actually sending a real signal.

Summary

Use `ps`/`top` to inspect, `kill` to stop (starting with the polite default signal, escalating to -9 only if needed), and `nohup`/`&` to keep long-running jobs alive after you disconnect.

Practice Exercises

- Find the PID of any running `bash` process using `ps aux | grep bash`.
- Start a long `sleep 300` command in the background using `&`, then check it with `jobs`.

CH 11 User and Group Management

Administering who can log in and what they belong to — essential for any multi-user or server environment.

Quick Reference

Command	Syntax	Description
useradd	`sudo useradd [options] name`	Create a new user account
userdel	`sudo userdel [-r] name`	Delete a user account
usermod	`sudo usermod [options] name`	Modify an existing user account
passwd	`passwd [user]`	Set or change a user's password
groupadd	`sudo groupadd name`	Create a new group
groupdel	`sudo groupdel name`	Delete a group

Command	Syntax	Description
groups	`groups [user]`	Show which groups a user belongs to
id	`id [user]`	Show UID, GID, and group memberships
whoami	`whoami`	Print the current logged-in username

Spotlight: useradd

Syntax

```
sudo useradd [options] username
```

Option	Meaning
`-m`	Create the user's home directory
`-s /bin/bash`	Set the user's default shell
`-G group1,group2`	Add the user to supplementary groups at creation

Example

```
sudo useradd -m -s /bin/bash -G sudo,developers priya
```

Expected Output

```
(no output; verify with `id priya`)
```

Use Case: Onboarding a new team member or service account on a shared server.

> **Common Mistake:** Forgetting -m, which leaves the new user without a home directory, breaking many default shell configurations.

> **Pro Tip:** On Debian/Ubuntu, prefer the friendlier `adduser` command, which prompts interactively and creates the home directory automatically.

Spotlight: usermod

Syntax

```
sudo usermod [options] username
```

Option	Meaning
`-aG group`	Append the user to an additional group (always pair -a with -G)
`-l newname`	Rename the login name
`-L / -U`	Lock / unlock the account's password

Example

```
sudo usermod -aG docker alice
```

Expected Output

```
(no output; alice must log out and back in for the change to take effect)
```

Use Case: Granting an existing user access to a new group, like adding them to 'docker' or 'sudo'.

> **Common Mistake:** Using `-G` without `-a` — this replaces all of the user's existing supplementary groups instead of adding to them.

> **Pro Tip:** Always double-check with `groups username` after any usermod change to confirm the result.

Summary

Creating a user is rarely just `useradd` — pair it with -m for a home directory and -G for group access, and always verify the result with `id`.

Practice Exercises

- Create a new user with a home directory and bash as the default shell.
- Add an existing user to a new group without removing their current group memberships.

CH 12 Networking Commands

Diagnosing connectivity, inspecting network configuration, and transferring data over a network.

Quick Reference

Command	Syntax	Description
ping	`ping host`	Test reachability of a host
ip a	`ip a`	Show network interfaces and IP addresses (modern replacement for ifconfig)
ifconfig	`ifconfig`	Legacy command to show/configure interfaces
ss	`ss -tulnp`	Show open sockets and listening ports (modern netstat)
netstat	`netstat -tulnp`	Legacy command to show network connections
curl	`curl [options] url`	Transfer data to/from a URL, print response
wget	`wget url`	Download a file from a URL
ssh	`ssh user@host`	Securely log in to a remote machine
scp	`scp file user@host:path`	Securely copy files over SSH
traceroute	`traceroute host`	Show the network path/hops to a host
dig / nslookup	`dig domain`	Query DNS records for a domain
hostname	`hostname`	Show or set the system's hostname

Spotlight: ssh

Syntax

```
ssh [options] user@host
```

Option	Meaning
`-p port`	Connect on a non-default port
`-i keyfile`	Use a specific private key for authentication
`-L local:host:remote`	Set up local port forwarding

Example

```
ssh -i ~/.ssh/id_rsa -p 2222 deploy@203.0.113.10
```

Expected Output

```
Welcome to Ubuntu 22.04.3 LTS ...
deploy@web-01:~$
```

Use Case: Logging into a remote server to deploy code, check logs, or run administrative commands.

> **Common Mistake:** Leaving private key files world-readable — SSH will refuse to use a key with overly open permissions until you `chmod 600` it.

> **Pro Tip:** Set up `~/.ssh/config` with a Host alias so you can just type `ssh myserver` instead of the full user@host string.

Spotlight: curl

Syntax

```
curl [options] url
```

Option	Meaning
`-I`	Fetch headers only (HEAD request)
`-o file`	Save the response body to a file
`-X POST`	Set the HTTP method
`-H 'Header: value'`	Add a custom header, e.g. for auth tokens

Example

```
curl -s -o /dev/null -w '%{http_code}\n' https://example.com
```

Expected Output

```
200
```

Use Case: Quickly checking whether an API or website is up and returning the expected status code.

> **Common Mistake:** Forgetting that curl follows redirects only when you pass -L — without it, a 301/302 response is shown as-is, not followed.

> **Pro Tip:** Use `curl -v` when debugging a failing request — it prints the full request and response headers.

Summary

`ping` and `traceroute` tell you if and how you can reach a host; `ss` and `netstat` tell you what's listening locally; `curl`/`wget` fetch data; `ssh`/`scp` connect and transfer securely.

Practice Exercises

- Check which ports are currently listening on your machine using `ss -tulnp`.
- Use `curl` to fetch just the HTTP status code of a website of your choice.

CH 13 Disk Usage and Storage

Inspecting how much space is used, where it's going, and managing mounted filesystems.

Quick Reference

Command	Syntax	Description
`df`	`df [-h] [path]`	Show free/used disk space per mounted filesystem
`du`	`du [-sh] path`	Show disk usage of files/directories
`mount`	`mount device dir`	Attach a filesystem to the directory tree
`umount`	`umount device|dir`	Detach a mounted filesystem
`lsblk`	`lsblk`	List block devices (disks and partitions) in a tree
`fdisk`	`sudo fdisk -l`	List or modify disk partition tables
`fsck`	`sudo fsck device`	Check and repair a filesystem
`blkid`	`sudo blkid`	Show filesystem type and UUID for each block device

Spotlight: df

Syntax

```
df [options] [path]
```

Option	Meaning
`-h`	Human-readable sizes (GB/MB instead of raw blocks)
`-T`	Show the filesystem type per mount

Example

```
df -h
```

Expected Output

```
Filesystem Size Used Avail Use% Mounted on
/dev/sda1 50G 32G 16G 67% /
```

Use Case: Checking whether a server is about to run out of disk space before it causes an outage.

> **Common Mistake:** Confusing `df` (space per filesystem/partition) with `du` (space used by specific files/directories) — they answer different questions.

> **Pro Tip:** A 100% full root partition can crash services silently — set up monitoring alerts well before that point, e.g. at 80%.

Spotlight: du

Syntax

```
du [options] [path]
```

Option	Meaning
`-s`	Summarize — show only the total for each argument
`-h`	Human-readable sizes
`--max-depth=1`	Limit how many directory levels deep to report

Example

```
du -sh /var/log/*
```

Expected Output

```
120M /var/log/nginx
45M /var/log/mysql
2.1M /var/log/apt
```

Use Case: Finding which specific directory is consuming disk space when `df` shows a partition nearly full.

> **Common Mistake:** Running plain `du` on a large directory without -s, which prints a line for every single subdirectory and floods your terminal.

> **Pro Tip:** Chain with sort to find the biggest offenders fast: `du -sh * | sort -rh | head -10`.

Summary

`df` answers 'how full is this partition', `du` answers 'what's using the space' — you'll usually use them together to hunt down disk usage problems.

Practice Exercises

- Check overall disk usage on your system with `df -h`.
- Find the three largest directories inside `/var/log` using `du` and `sort`.

CH 14 Package Management (APT, DNF, YUM, Pacman)

Installing, updating, and removing software differs by distribution family, but each package manager follows the same basic pattern: update the index, then install, remove, or search.

Quick Reference

Command	Syntax	Description
apt update	sudo apt update	Debian/Ubuntu: refresh the package index
apt install	sudo apt install pkg	Debian/Ubuntu: install a package
apt remove	sudo apt remove pkg	Debian/Ubuntu: remove a package (keep config)
apt purge	sudo apt purge pkg	Debian/Ubuntu: remove a package and its config
dnf install	sudo dnf install pkg	Fedora/CentOS Stream: install a package
yum install	sudo yum install pkg	Older RHEL/CentOS: install a package
pacman -S	sudo pacman -S pkg	Arch Linux: install a package
pacman -Syu	sudo pacman -Syu	Arch Linux: refresh index and upgrade all packages
dpkg	dpkg -i file.deb	Install a local .deb package file directly
rpm	rpm -ivh file.rpm	Install a local .rpm package file directly

Spotlight: apt

Syntax

```
sudo apt [subcommand] [package]
```

Option	Meaning
update	Refresh the list of available packages and versions
upgrade	Install newer versions of all upgradable packages
install pkg	Install a specific package and its dependencies
autoremove	Remove packages that were installed as dependencies but are no longer needed

Example

```
sudo apt update && sudo apt install nginx
```

Expected Output

```
Reading package lists... Done
The following NEW packages will be installed:
nginx nginx-common
...
Setting up nginx ...
```

Use Case: The standard first two commands run on any fresh Debian/Ubuntu server before installing anything.

> **Common Mistake:** Running `apt install` without `apt update` first, which can install an outdated version or fail to find a newer package.

> **Pro Tip:** On Fedora/CentOS the equivalent single command is `sudo dnf install pkg` — DNF checks for updates automatically.

Summary

Same idea, different name across families: apt (Debian/Ubuntu), dnf/yum (Fedora/RHEL/CentOS), pacman (Arch). Always update the index before installing, and use the package manager rather than downloading binaries by hand whenever possible.

Practice Exercises

- On a Debian/Ubuntu system, update the package index and then search for a package named 'curl'.
- Identify which package manager your own distribution uses and list its install command.

CH 15 Environment Variables and Shell Configuration

Environment variables configure how programs and the shell itself behave, and shell configuration files control what gets set up every time you open a terminal.

Quick Reference

Command	Syntax	Description
export	`export VAR=value`	Set an environment variable for the current shell and its children
env	`env`	List all current environment variables
echo $VAR	`echo $HOME`	Print the value of a specific variable
unset	`unset VAR`	Remove an environment variable
alias	`alias ll='ls -lah'`	Create a shortcut for a longer command
source	`source ~/.bashrc`	Reload a shell config file into the current session
.bashrc	`(file)`	Bash config loaded for every new interactive terminal
.bash_profile	`(file)`	Bash config loaded once at login (login shells)

Spotlight: export

Syntax

```
export VARIABLE=value
```

Option	Meaning
`VARIABLE=value`	No spaces around the equals sign
`$PATH:/new/dir`	Append to an existing variable rather than overwriting it

Example

```
export PATH=$PATH:/opt/myapp/bin
```

Expected Output

```
(no output; confirm with `echo $PATH`)
```

Use Case: Making a custom tool's binary available from any directory without typing the full path.

> **Common Mistake:** Overwriting PATH entirely (`export PATH=/opt/myapp/bin`) instead of appending, which breaks every standard command until you fix it.

> **Pro Tip:** Put export lines in `~/.bashrc` (or `~/.zshrc`) so they persist across terminal sessions instead of disappearing when you close the window.

Summary

Environment variables configure the current session; `.bashrc`/`.bash_profile` make those settings permanent by running automatically on shell startup.

Practice Exercises

- Create a temporary alias called `gs` for `git status`, then confirm it works.
- Print the value of your PATH variable and count how many directories it contains.

CH 16 Input/Output Redirection and Pipes

Every process has three standard streams — input, output, and error — and Linux lets you redirect or chain them freely between commands and files.

Quick Reference

Operator	Syntax	Description
>	`cmd > file`	Redirect standard output to a file (overwrite)
>>	`cmd >> file`	Redirect standard output to a file (append)
<	`cmd < file`	Use a file as standard input
|	`cmd1 | cmd2`	Pipe the output of one command into the input of another
2>	`cmd 2> file`	Redirect standard error only
&>	`cmd &> file`	Redirect both standard output and standard error
tee	`cmd | tee file`	Write output to a file and print it at the same time
xargs	`cmd | xargs cmd2`	Convert piped input into arguments for another command

Spotlight: | (pipe)

Syntax

```
command1 | command2
```

Option	Meaning
`stdout -> stdin`	The output of command1 becomes the input of command2
`chainable`	Pipes can be chained: cmd1 | cmd2 | cmd3

Example

```
ps aux | grep nginx | wc -l
```

Expected Output

```
3
```

Use Case: Composing simple tools together to answer a specific question — here, counting nginx processes.

> **Common Mistake:** Expecting a pipe to work with commands that don't read from standard input at all — check the command's docs first.

> **Pro Tip:** Build long pipelines incrementally: run the first command alone, confirm the output, then add the next stage one at a time.

Spotlight: tee

Syntax

```
command | tee [-a] file
```

Option	Meaning
`-a`	Append to the file instead of overwriting it

Example

```
build.sh | tee build.log
```

Expected Output

```
(build output prints to the terminal AND is saved into build.log)
```

Use Case: Watching a long-running command's output live while also saving a copy for later review.

> **Common Mistake:** Forgetting that without -a, tee overwrites the log file every time — useful for a fresh run, risky if you meant to append.

> **Pro Tip:** Use `tee -a` inside a loop or repeated script run to build up a cumulative log file over time.

Summary

Redirection controls where a stream goes (file vs. terminal); pipes connect streams between commands. Together they're the foundation of nearly every one-liner in this book.

Practice Exercises

- Redirect the output of `ls -la` into a file called `filelist.txt`.
- Use a pipe to count how many lines in `/etc/passwd` mention the word 'bash'.

CH 17 Bash Scripting Basics

A shell script is just a text file of commands, run in sequence, with support for variables, conditionals, and loops — perfect for automating repetitive tasks.

Quick Reference

Element	Syntax	Description
`#!/bin/bash`	`(first line)`	Shebang — tells the OS which interpreter to use
`VAR=value`	`name="Alice"`	Assign a variable (no spaces around =)
`$VAR / ${VAR}`	`echo $name`	Reference a variable's value
`if / then / fi`	`if [ cond ]; then ...; fi`	Conditional branching
`for`	`for i in 1 2 3; do ...; done`	Loop over a list of values
`while`	`while [ cond ]; do ...; done`	Loop while a condition is true
`function`	`myFunc() { ...; }`	Define a reusable function
`read`	`read name`	Read a line of input into a variable
`chmod +x`	`chmod +x script.sh`	Make a script file executable

Spotlight: Sample Script

Syntax

```
#!/bin/bash
```

Option	Meaning
`$1, $2 ...`	Positional arguments passed to the script

`$#`	Number of arguments passed
`exit 0`	End the script with a success status code

Example

```
#!/bin/bash
# backup.sh - archive a directory with a timestamp
SOURCE=$1
DEST="backup_$(date +%Y%m%d).tar.gz"

if [ -z "$SOURCE" ]; then
echo "Usage: ./backup.sh <directory>"
exit 1
fi

tar -czf "$DEST" "$SOURCE"
echo "Backup saved to $DEST"
```

Expected Output

```
$ ./backup.sh project/
Backup saved to backup_20260729.tar.gz
```

Use Case: Automating a repeated task — here, timestamped backups — into a single reusable command.

Common Mistake: Forgetting `chmod +x script.sh` before running it with `./script.sh`, which results in a 'Permission denied' error.

Pro Tip: Always quote variables ("$SOURCE" not $SOURCE) to avoid the script breaking on filenames containing spaces.

Summary

Every script starts with a shebang, uses variables without spaces around =, and needs execute permission before it can be run directly.

Practice Exercises

- Write a script that prints 'Hello, ' using a name passed as the first argument.
- Modify the backup.sh example to also print an error if the given directory does not exist.

CH 18 System Information Commands

Quickly answering 'what am I running, and how healthy is it' — hardware, kernel, uptime, and resource usage at a glance.

Quick Reference

Command	Syntax	Description
`uname -a`	`uname -a`	Show kernel name, version, and architecture
`hostnamectl`	`hostnamectl`	Show hostname and OS details (systemd distros)
`uptime`	`uptime`	Show how long the system has been running and load average

Command	Syntax	Description
`free -h`	`free -h`	Show memory and swap usage in human-readable form
`lscpu`	`lscpu`	Show detailed CPU architecture information
`lsusb`	`lsusb`	List connected USB devices
`lspci`	`lspci`	List connected PCI devices (GPUs, network cards, etc.)
`date`	`date`	Show or set the current system date and time
`cal`	`cal`	Display a calendar in the terminal

Spotlight: uname

Syntax

```
uname [options]
```

Option	Meaning
`-a`	All information: kernel name, hostname, version, architecture
`-r`	Kernel release version only
`-m`	Machine hardware architecture only (e.g. x86_64)

Example

```
uname -a
```

Expected Output

```
Linux web-01 6.5.0-28-generic #29-Ubuntu SMP x86_64 GNU/Linux
```

Use Case: Confirming kernel version and architecture before installing software that has specific compatibility requirements.

Common Mistake: Assuming `uname -a` tells you the distribution name — it only reports kernel information, not distro (use `hostnamectl` or `/etc/os-release` for that).

Pro Tip: Check `/etc/os-release` with `cat` for a clean, script-friendly summary of the distro name and version.

Spotlight: free

Syntax

```
free [options]
```

Option	Meaning
`-h`	Human-readable units (GB/MB instead of raw KB)
`-s N`	Repeat the report every N seconds

Example

```
free -h
```

Expected Output

```
total used free shared buff/cache available
Mem: 15Gi 4.2Gi 1.1Gi 312Mi 10Gi 10Gi
```

Use Case: Checking whether a server is under memory pressure before diagnosing performance issues.

Common Mistake: Panicking at a low 'free' number without checking 'available' — Linux uses spare RAM for disk caching, which is reclaimed automatically when needed.

Pro Tip: Focus on the 'available' column, not 'free' — it's the more accurate picture of memory truly available to new processes.

Summary

These commands are your first stop when diagnosing 'why is this server slow' or confirming what you're running before installing new software.

Practice Exercises

- Check your system's total and available memory using `free -h`.
- Find your kernel version and CPU architecture using `uname`.

CH 19

Top 50 Essential Linux Commands — Cheat Sheet

A single-page reference of the commands used most often day to day. For full syntax, options, and examples, refer back to the relevant chapter.

Command	Description
`pwd`	Print current directory
`cd`	Change directory
`ls -la`	List all files, long format
`mkdir -p`	Create nested directories
`touch`	Create empty file
`rm -rf`	Force-remove recursively
`cp -r`	Copy directories recursively
`mv`	Move or rename
`cat`	Print file contents
`less`	Page through a file
`head -n`	Show first N lines
`tail -f`	Follow a file live
`find -name`	Search files by name
`locate`	Fast indexed file search
`grep -rn`	Search text recursively
`sed 's/a/b/'`	Find and replace text
`awk '{print $1}'`	Print a column
`sort`	Sort lines
`uniq -c`	Count duplicate lines
`wc -l`	Count lines
`chmod 755`	Set rwx permissions
`chown user:group`	Change ownership
`tar -czvf`	Create a .tar.gz archive
`tar -xzvf`	Extract a .tar.gz archive
`ps aux`	List all processes
`top / htop`	Live process monitor

Command	Description
`kill -9`	Force-kill a process
`jobs / bg / fg`	Manage background jobs
`useradd -m`	Create user with home dir
`usermod -aG`	Add user to a group
`passwd`	Change a password
`ping`	Test host reachability
`ip a`	Show IP addresses
`ss -tulnp`	Show listening ports
`curl -I`	Fetch HTTP headers
`ssh user@host`	Remote login
`scp file host:path`	Secure copy over SSH
`df -h`	Disk space per filesystem
`du -sh`	Disk usage of a directory
`lsblk`	List block devices
`apt update && apt install`	Debian/Ubuntu package install
`dnf install`	Fedora/CentOS package install
`pacman -S`	Arch package install
`export VAR=value`	Set environment variable
`alias`	Create a command shortcut
`source ~/.bashrc`	Reload shell config
`cmd > file`	Redirect output (overwrite)
`cmd >> file`	Redirect output (append)
`cmd1 | cmd2`	Pipe between commands
`xargs`	Turn piped input into arguments
`chmod +x script.sh`	Make a script executable

CH 20 Interview Questions, Practice & Final Revision

Common Interview Questions

Q1. What is the difference between a hard link and a symbolic link?

A hard link points directly to the same inode as the original file and keeps working even if the original is deleted; a symbolic link is a separate file that stores a path to the original, and breaks if that path disappears.

Q2. What's the difference between `su` and `sudo`?

`su` switches you to another user's shell entirely (typically requiring that user's password); `sudo` runs a single command with elevated privileges using your own password, then returns you to your normal shell.

Q3. How do you find which process is using a specific port?

Use `ss -tulnp | grep :PORT` (or the older `netstat -tulnp | grep :PORT`) to see the process name and PID bound to that port.

Q4. What does `chmod 644` mean?

Owner gets read+write (6), group gets read-only (4), others get read-only (4) — a typical permission set for a regular, non-executable file.

Q5. What is the difference between `>` and `>>`?

`>` overwrites the target file with new output; `>>` appends the new output to the end of the existing file.

Q6. How would you find and delete all .tmp files older than 7 days?

`find /path -name '*.tmp' -mtime +7 -exec rm {} \;` or the faster `find /path -name '*.tmp' -mtime +7 | xargs rm`.

Q7. What's the difference between a process and a thread, and how do you see both?

A process is an independent running program with its own memory space; threads run within a process and share its memory. `ps` and `top` show processes; `top -H` or `ps -eLf` reveal individual threads.

Q8. What does `kill -9` actually do, and why should it be a last resort?

It sends SIGKILL, forcing the OS to terminate the process immediately without letting it clean up open files or child processes — a plain `kill` (SIGTERM) gives the process a chance to shut down gracefully first.

Q9. How do environment variables set in a script differ from ones set interactively?

Both work the same way at runtime, but variables set inside a script (without `export`) are local to that script's shell and are not passed to child processes unless exported.

Q10. What is the purpose of the /proc filesystem?

It's a virtual filesystem that exposes real-time kernel and process information as if it were files — for example, /proc/cpuinfo and /proc/<pid>/status — without those files actually existing on disk.

Q11. How would you check disk space and find what's consuming it?

Start with `df -h` to see which partition is full, then use `du -sh /path/* | sort -rh | head` to identify the largest directories inside it.

Q12. What's the difference between `apt upgrade` and `apt dist-upgrade` (or `full-upgrade`)?

`apt upgrade` updates packages without removing any existing ones, even if that means skipping an update; `full-upgrade` will also add or remove packages as needed to complete the upgrade, including handling changed dependencies.

Hands-On Practice Exercises

- Write a one-line pipeline that finds the five largest files in your home directory.
- Create a user, add them to a new group, and confirm the group membership with `id`.
- Write a bash script that backs up a given directory into a timestamped .tar.gz file.
- Use `grep`, `sort`, and `uniq -c` together to find the most common IP address in a web server log.
- Set up an SSH key pair and use it to log into a remote server without a password prompt.
- Diagnose a 'disk full' scenario: use `df -h` to find the full partition, then `du` to find the cause.

Final Revision Map

Category	Key Commands
Navigation	`pwd, cd, ls`
File Ops	`touch, mkdir, cp, mv, rm`
Viewing Files	`cat, less, head, tail -f`
Searching	`find, locate, grep`
Text Processing	`sed, awk, cut, sort, uniq, wc`
Permissions	`chmod, chown, umask`
Archiving	`tar, zip, gzip`
Processes	`ps, top, kill, jobs`
Users	`useradd, usermod, passwd`
Networking	`ping, ssh, curl, ss`
Storage	`df, du, lsblk`
Packages	`apt, dnf, pacman`
Shell Config	`export, alias, source`
Redirection	`> >> < | tee xargs`
Scripting	`#!/bin/bash, if, for, while, functions`